Class

- Class is basis in OOP.
- Class defines new data type.
  - Ex: int a, b;
  - struct student
    {
    };
    class student
    {
    }
    Student s;
- It can be used to create objects of that type.
- Thus, a class is a template for an object, and an object is an instance of a class.

  object=instance (both are same)

Class

A class is declared by use of the **class keyword.**

A simplified general form of a **class definition is shown here:**

```
modifier class classname
 {
  type instance-variable1;
  type instance-variable2;
  // ...
  type instance-variableN;
  type methodname1(parameter-list) {
        // body of method
  }
   // ...
  type methodnameN(parameter-list) {
        // body of method
  }
}
```

- ✔The data, or variables, defined within a **class are called *instance variables.***
- ✔***The code is*** contained within *methods.*
- ✔*Collectively, the methods and variables defined within a class* are called *members of the class.*
- ✔*In most classes, the instance variables are* accessed by the methods defined in that class.

Example

```
class Box {
    double width;
    double height;
    double depth;
}
```

Declaration of object

Box mybox = new Box();

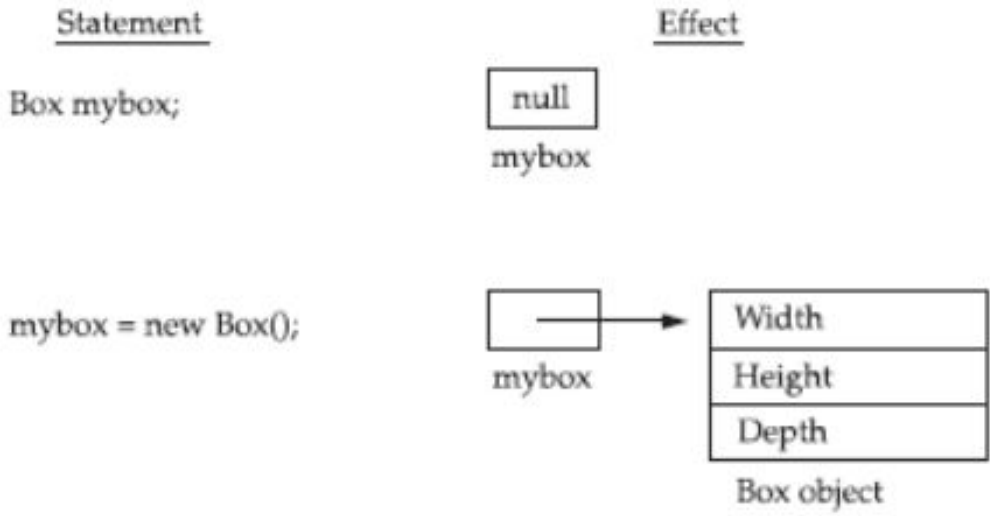
This statement combines the two steps just described. It can be rewritten like this to show each step more clearly:

class-var = new classname ( );

Box mybox; // declare reference to object

mybox = new Box(); // allocate a Box object

- constructor



```
/* A program that uses the Box class. Call this file BoxDemo.java */
class Box {
      double width;
      double height;
      double depth;
}
// This class declares an object of type Box.
class BoxDemo {
        public static void main(String args[]) {
                        Box mybox = new Box();
                        double vol;
                        // assign values to mybox's instance variables
                        mybox.width = 10;
                        mybox.height = 20;
                        mybox.depth = 15;
                        // compute volume of box
                        vol = mybox.width * mybox.height * mybox.depth;
                        System.out.println("Volume is " + vol);
                }
            }
```

```
// This program declares two Box objects.
class Box {
        double width;
        double height;
        double depth;
}
class BoxDemo2 {
        public static void main(String args[]) {
                                Box mybox1 = new Box();
                                 Box mybox2 = new Box();
                               double vol;
                                    // assign values to mybox1's instance variables
                                    mybox1.width = 10;
                                    mybox1.height = 20;
                                    mybox1.depth = 15;
                                    /* assign different values to mybox2's
                                    instance variables */
                                    mybox2.width = 3;
                                    mybox2.height = 6;
                                    mybox2.depth = 9;
                                    // compute volume of first box
                                    vol = mybox1.width * mybox1.height * mybox1.depth;
                                    System.out.println("Volume is " + vol);
                                    // compute volume of second box
                                    vol = mybox2.width * mybox2.height * mybox2.depth;
                                    System.out.println("Volume is " + vol);
                        }
}
```

Introducing Methods

This is the general form of a method:

modifier type method_name(parameter-list) {

// body of method

}

Methods that have a return type other than void return a value to the calling routine using the following form of the return statement:

- *return value;*

Here, value is the value returned.

-Example, returning value

-Example, passing parameters

•✅

Method Type	Returns Value	Takes Parameters
void display()	❌ No	❌ No
void display(int x)	❌ No	✅ Yes
int sum()	✅ Yes	❌ No
int sum(int a, int b)	✅ Yes	✅ Yes

- Method without Parameters and without Return Value
- Method with Parameters
- Method with Return Value
- Method with Parameters and Return Value

Adding a Method that takes Parameters

A parameterized method can operate on a variety of data and/or be used in a number of slightly different situations.

For example. Here is a method that returns the square of the number 10:

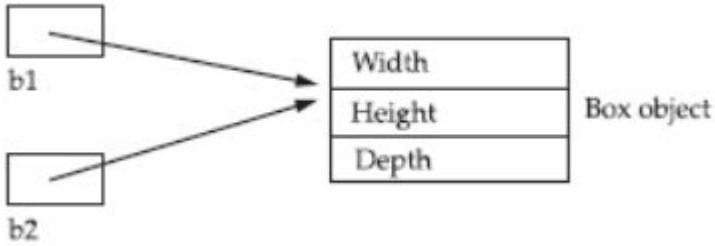
```
int square()
{
    return 10 * 10;
}

int square(int i)
{
    return i * i;
}
```

Now, **square( ) will return the square of whatever value it is called with.**

Assigning Object Reference Variables

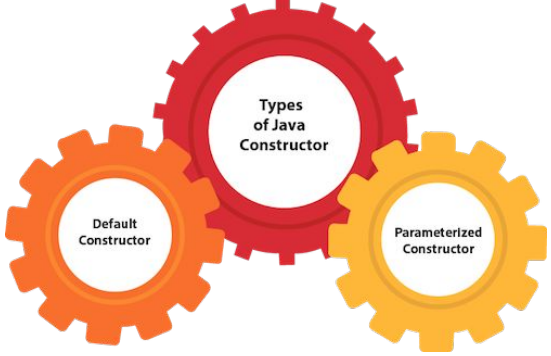
```
Box b1 = new Box();
Box b2 = b1;
```




```
Box b1 = new Box();
Box b2 = b1;
// ...
b1 = null;
```

A reference variable is a variable that stores the memory address (reference) of an object, rather than storing the actual object itself. It is used to access the object's data members and methods.

Constructors



- Java allows objects to initialize themselves when they are created. This automatic initialization is performed through the use of a constructor.
- A *constructor initializes an object immediately upon creation. It has the same name as* the class in which it resides and is syntactically similar to a method.
- Once defined, the constructor is automatically called immediately after the object is created.
- Constructors look a little strange because they have no return type, not even **void.**
- **This is because the implicit return type of a class' constructor is the class type** itself.

- Ex: default, parameterized

- If there is no constructor in a class, compiler automatically creates a default constructor.



this keyword

- Refers to the current object
- **Key Points**
  - this refers to the **current object**.
  - It can be used only inside **instance methods** and **constructors**.
  - It **cannot** be used inside a **static method** because static methods do not belong to a specific object.
  - **this() must be the first statement in a constructor.**

Use	Description
this.variable	Refers to the current object's instance variable.
this.method()	Calls the current class method.
this()	Calls another constructor in the same class (constructor chaining).
this	Passes the current object as an argument.
return this	Returns the current object.

Garbage collection

- Since objects are dynamically allocated by using the **new operator, deallocation** is done automatically.
- In some languages, such as C++, dynamically allocated objects must be manually released by use of a **delete operator.**
- **Java takes a different approach; it handles deallocation for you automatically.**
- **Garbage Collection (GC)** is the automatic process of removing **unused or unreachable objects** from memory (heap) to free memory. It is performed automatically by the **Java Virtual Machine (JVM)**.

Method Overloading

- In Java it is possible to define two or more methods within the same class that share the same name, as long as their parameter declarations are different.
- When this is the case, the methods are said to be overloaded, and the process is referred to as ***method overloading***.
- Method overloading is one of the ways that Java supports polymorphism.
- When an overloaded method is invoked, Java uses the type and/or number of arguments as its guide to determine which version of the overloaded method to actually call. Thus, overloaded methods must differ in the type and/or number of their parameters.
- While overloaded methods may have different return types, the return type alone is insufficient to distinguish two versions of a method.

Automatic type conversion - if no exact match is found

Using Objects as Parameters

- Using **objects as parameters** means passing an object of a class to a method.
- Instead of passing individual values, you pass the entire object, allowing the method to access all of its data members.

```
// Objects may be passed to methods.
class Test {
      int a, b;
      Test(int i, int j) {
        a = i;
        b = j;
      }
      // return true if o is equal to the invoking object
      boolean equals(Test o) {
        if(o.a == a && o.b == b) return true;
                else return false;
        }
}
class PassingObject2 {
      public static void main(String args[]) {
        Test ob1 = new Test(100, 22);
        Test ob2 = new Test(100, 22);
        Test ob3 = new Test(-1, -1);
        System.out.println("ob1 == ob2: " + ob1.equals(ob2));
        System.out.println("ob1 == ob3: " + ob1.equals(ob3));
      }
}
```

Call-by-value, call-by-reference

- In JAVA, there are two ways to pass an argument to a subroutine.
- The first way is *call-by-value.:*
  - *This approach copies the value of an argument into the formal* parameter of the subroutine. Therefore, changes made to the parameter of the subroutine have no effect on the argument.
- The second way an argument can be passed is *call-by-reference:*
  - In this approach, a reference to an argument (not the value of the argument) is passed to the parameter. Inside the subroutine, this reference is used to access the actual argument specified in the call. This means that changes made to the parameter will affect the argument used to call the subroutine.
- When you pass a primitive type to a method, it is passed by value.
- When you pass an object to a method, the situation changes dramatically, because objects are passed by what is effectively call-by-reference .
-

Returning Objects

- A method can return any type of data, including class types that you create.
- When a method returns an object, it returns the **reference** to that object.

Use of final

- The **final keyword** in Java is used to restrict the user.
- The java final keyword can be used in many contexts.
- Final can be:
  - variable
  - method
  - class



- A final variable that have no value is called blank final variable or uninitialized final variable.
- It can be initialized inside the class/ constructor only.

Introducing Access Control

	Private	No Modifier	Protected	Public
Same class	Yes	Yes	Yes	Yes
Same package subclass	No	Yes	Yes	Yes
Same package non-subclass	No	Yes	Yes	Yes
Different package subclass	No	No	Yes	Yes
Different package non-subclass	No	No	No	Yes

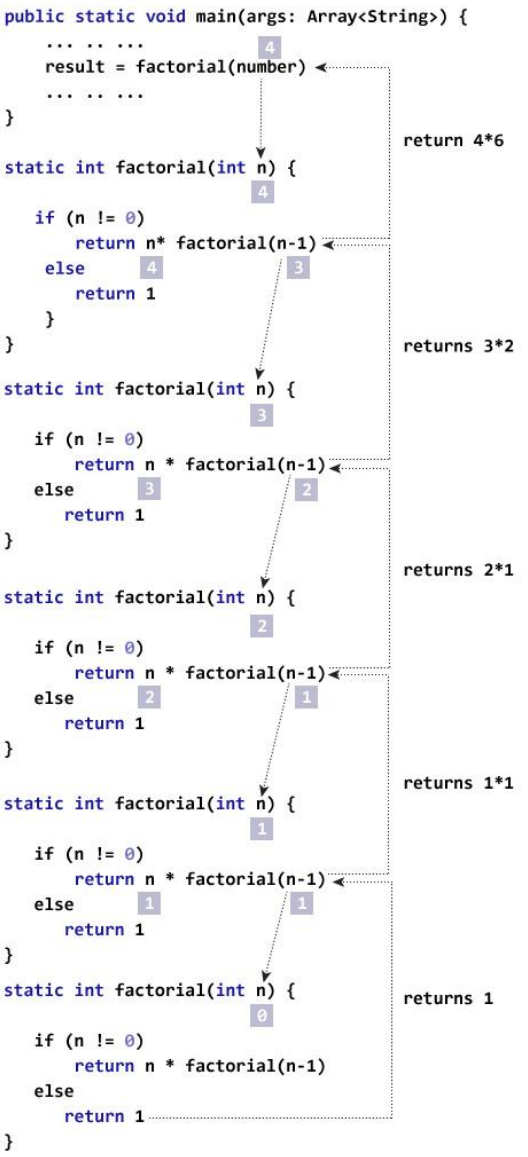
- Now we can understand why main() has always been preceded by the public modifier. It is called from outside the program—that is, by the Java run-time system.
- When no access modifier is used, then by default the member of a class is public (default) within its own package, but cannot be accessed outside of its package.

Recursion

- *Recursion is the process of defining something in terms of itself.*
- A method that calls itself is said to be *recursive.*
- Ex: Recursion_Fact.java

- There are two cases:
- **Base Case**: This is the condition that stops the recursion. It prevents infinite recursion by providing a simple, direct answer for a specific case.
- **Recursive Case**: This is the part where the method calls itself with a modified argument, progressively working towards the base case.

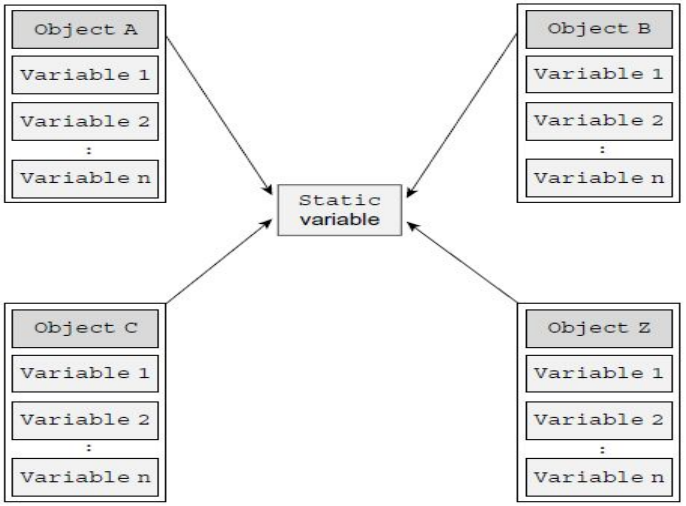
```
factorial(5)
   factorial(4)
      factorial(3)
         factorial(2)
            factorial(1)
               return 1
            return 2*1 = 2
         return 3*2 = 6
      return 4*6 = 24
   return 5*24 = 120
```



Use of static

- There will be times when you will want to define a class member that will be used independently of any object of that class.
- To create such a member, precede its declaration with the keyword **static.**
- You can declare both methods and variables to be **static.**
- **The most common example of a static member is main( ). main( ) is declared as static because it must be called before any objects exist.**

Default values for static and non-static variables are same.
primitive integers(long, short etc): 0
primitive floating points(float, double): 0.0
boolean: false
object references: null



- Variables declared as **static are, essentially, global variables**
- **All instances (objects) of the class** share the same **static variable.**
- Methods declared as **static have several restrictions:**
    - They can only directly call other **static methods.**
    - They can only directly access **static data.**
    - They cannot refer to **this or super in any way**

- We can also create static block. **Static block gets executed exactly once, when the class is first loaded.**

- If you wish to call a **static method from outside** its class, you can do so using the following general form:

  classname.method( )

  Or

  Object.method()

  Ex: UseStatic.java

  StaticVariableDemo.java

  StaticByName.java

Static memory is allocated during the compile time. Dynamic Memory allocation is done during run time.

- **Why a static method can't call a non-static method?**
  - Now that we know the constraints of using static in our methods, let's find out more on why a static method can't call a static method?
  - Simple answer — ***Compile-time or Early binding***
  - Usage of **static** keyword instructs the compiler to store the program instructions and variables into a physical memory. Compile time binding is done before loading the program into memory.
  - Whereas, a non-static method/variables are examples of Run-time or Late binding, where an object is required to be created before it is called.

Static Method vs. Instance Method

Static Methods	Instance Methods
Can be called without the objects,	Require an object creation to call an instance method
Associated with the class	Associated with the objects
Can access only static attributes.	Can access all attributes of a class
Requires to be declared using static keyword	Doesn’t require any keyword

Q) Can we execute a program without main() method?

Ans) Yes, one of the way is static block but in previous version of JDK not in JDK 1.7.

```
class A3{
  static{
  System.out.println("static block is invoked");
  System.exit(0);
  }
}
```

```
D:\Materials\JAVA Material\Unit 1\Class 2017-2018>javac Without.java

D:\Materials\JAVA Material\Unit 1\Class 2017-2018>java Without
Error: Main method not found in class Without, please define the main method as:
   public static void main(String[] args)
```

Test it Now

```
Output:static block is invoked (if not JDK7)
```

In JDK7 and above, output will be:

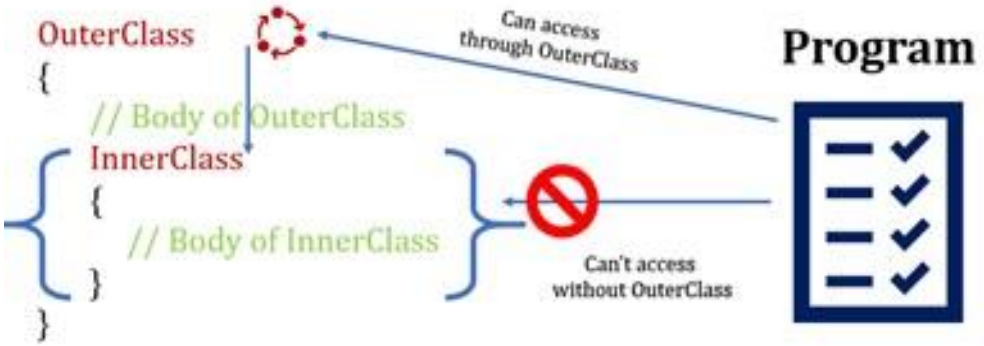
```
Output:Error: Main method not found in class A3, please define the main method as:
public static void main(String[] args)
```

Nested Classes:

- A nested class is one that is declared entirely in the body of another class or interface.
- The class that contains the other class is known as the **outer class** and the contained class is known as **inner class**.

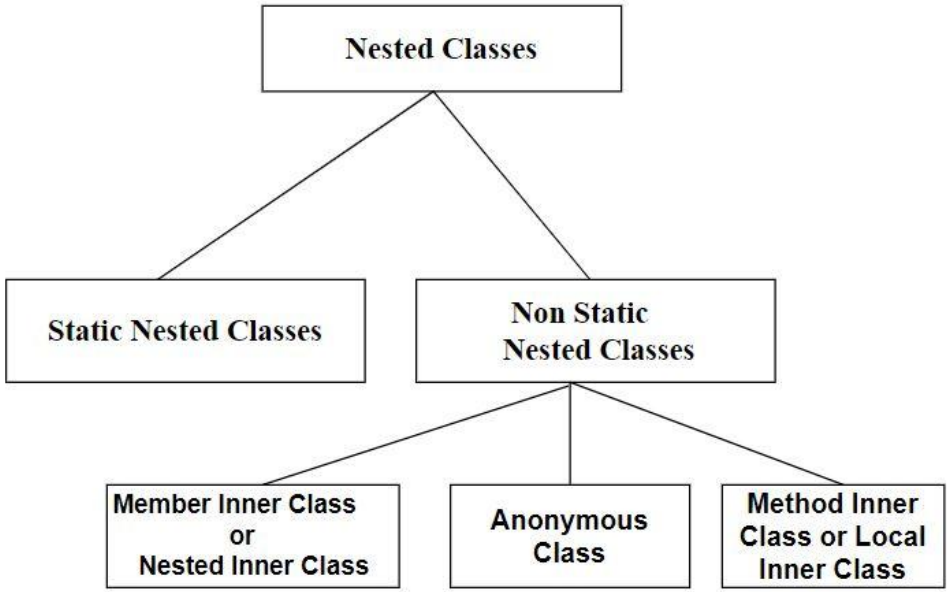
```
OuterClass
{
    // Body of OuterClass
    InnerClass
    {
     /// Body of InnerClass
    }
}
```

- The class which is nested, exists only as long as the enveloping class exists.
- So, the scope of inner class is limited to the scope of enveloping class.



- Nested classes enables us to logically group classes that are only used in one place, thus this increases the use of encapsulation

- Nested classes are divided into two categories:
  - **static nested class:** Nested classes that are declared static are called static nested classes.
  - **Nested non-static classes or inner classes** which may be member inner class, local class and anonymous class.



Static Nested Classes

- In Java, a static nested class is a nested class that is declared with the static keyword.
- Static classes can be accessed with outer class name.

Member Inner Class

- If the nested class is **not static**, it is called an **Inner Class**.

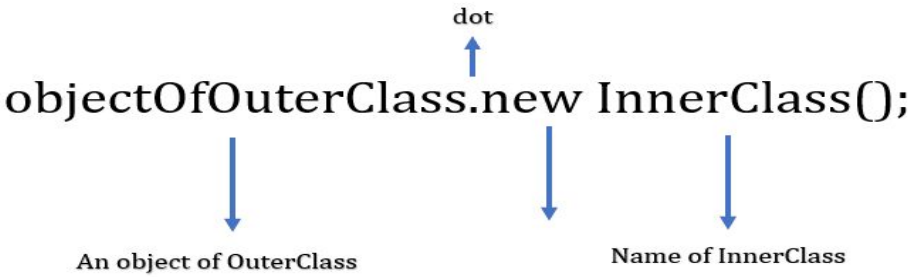
```
class OuterClass
{
    // A member inner class named Inner.
    class InnerClass
    {
      ........
    }
}
```

- We can't create an object or instance of a member inner class directly.
- The object/instance of a member inner class totally depends on the object of the outer class.

```
OuterClass oc_obj = new OuterClass();
InnerClass objOfInner = oc_obj.new InnerClass();
```

Member Inner Class

- We can’t create an object or instance of a member inner class directly. The object/instance of a member inner class totally depends on the object of the outer class.
- Unlike a normal class, the nested class has a different way to create an object or instance.



OuterClass objOfOuter = new OuterClass();

InnerClass objOfInner = objOfOuter.new InnerClass();

Anonymous Inner Class

- A class that has no name and whose object is created at the same time is called an anonymous inner class.
- It is commonly used to:
  - Override a method of a **class**
  - Implement a method of an **interface**
  - Provide a one-time implementation
- An anonymous class is defined by operator *new* followed by class name it extends, argument list for the constructor of super class, and then, the anonymous class body.

class-name obj=new *class-name* ([argument list]){class body};

- Java Anonymous inner class can be created in two ways:
  - Class (may be abstract or concrete).
  - Interface

Method Inner Class or Local Inner Class

- A class i.e., created inside a method or a block, is called local inner class in Java.
- Block may be *method or if clause or loop.*

Strings

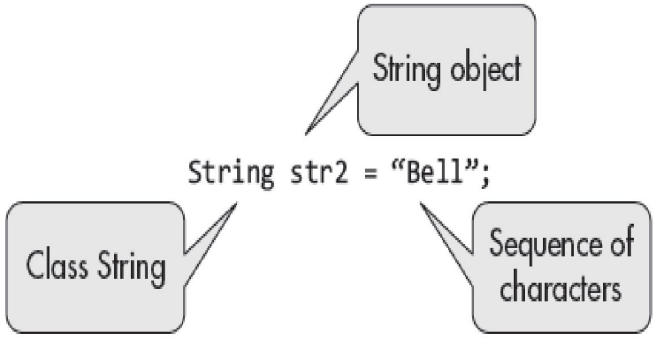
- A string is a sequence of characters.
- In Java, string is an object of class, and therefore there is no automatic appending of null character by the system.
- In Java, there are three classes that can create strings and process them with nearly similar methods.
  - class String
  - class StringBuffer
  - class StringBuilder
- All the three classes are part of java.lang package.

In C: char name[] = "Java"; - Internally -> J a v a \0
In JAVA - J a v a -There is **no need for '\0'** at the end.

- **Class String:**
  - In the case of String class, an object may be created as
    - String str1=”abcd”;
    - String str2=”bell”;
- and these are equivalent to char s1={‘a’,’b’,’c’,’d’}

  and

  char s2={‘b’,’e’,’l’,’l’}



Objects of class String may also be created by operator new

```
String str=new String(“abcd”);
```

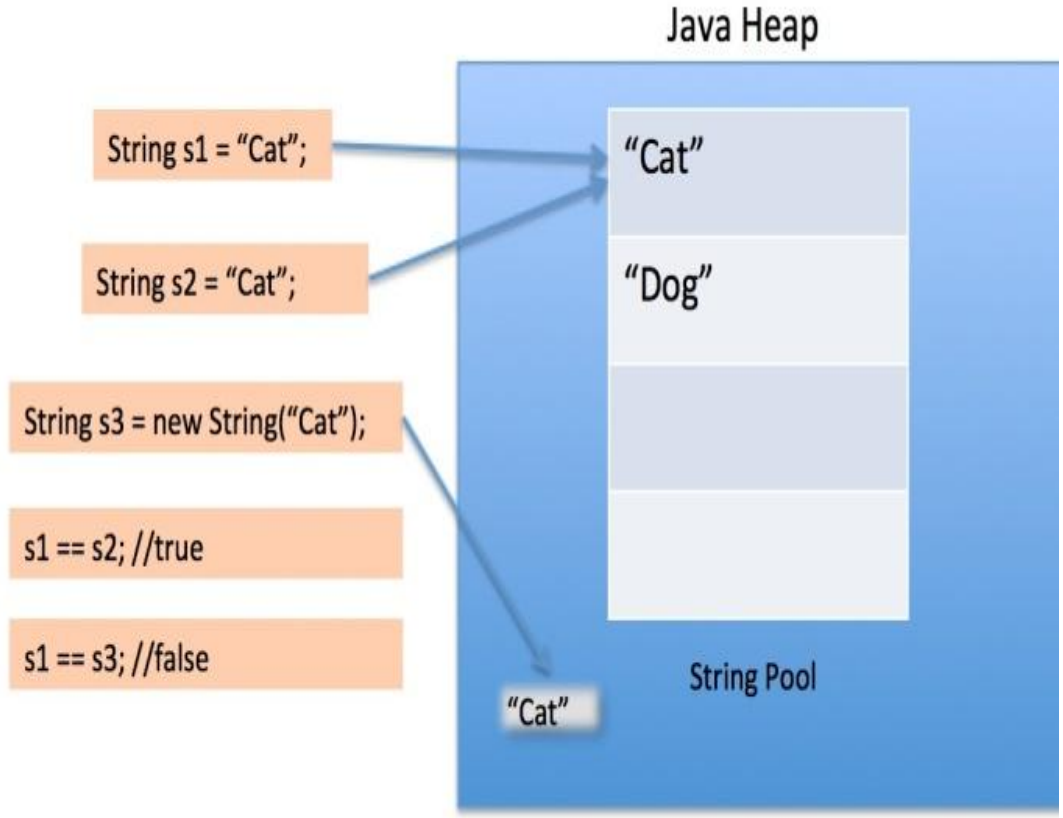
In the case of other two classes, the objects can be created only by the operator new as

```
StringBuffer sb=new StringBuffer(“PVPSIT”);
StringBuilder sb=new StringBuilder(“PVPSIT”);
```

Storage of Strings

- The objects of class String have a special storage facility, which is not available to objects of other two String classes or to objects of any other class.
- The memory allocated to a Java program is divided into two segments:

  (i) Stack (ii) Heap



- Within the heap, there is a memory segment called **'String Constant Pool (SCP)**'.
- The String class objects can be created in two different ways:

  String s1 = "Cat";

  String strz = new String("Cat");

String vs StringBuffer vs StringBuilder

Property	String	StringBuffer	StringBuilder
Storage area	String Constant Pool and Heap	Heap	Heap
Object mutability	Immutable (not changeable)	Mutable (changeable)	Mutable (changeable)
Thread safety	Safe	Safe (synchronized)	Not thread-safe
Performance	Fast	Relatively slow	Fast

Interface CharSequnece:

- It is an interface in java.lang package.
- It is implemented by several classes including the classes String, StringBuffer, and StringBuilder.

Modifier	Method Name
char	charAt(int index): Returns the char value at the specified index.
int	length(): Returns the length of this character sequence.
CharSequence	subSequence(int start, int end): Returns a new CharSequence that is a subsequence of this sequence.
String	toString(): Returns the String representation of the character sequence. toString() simply returns the same String content.

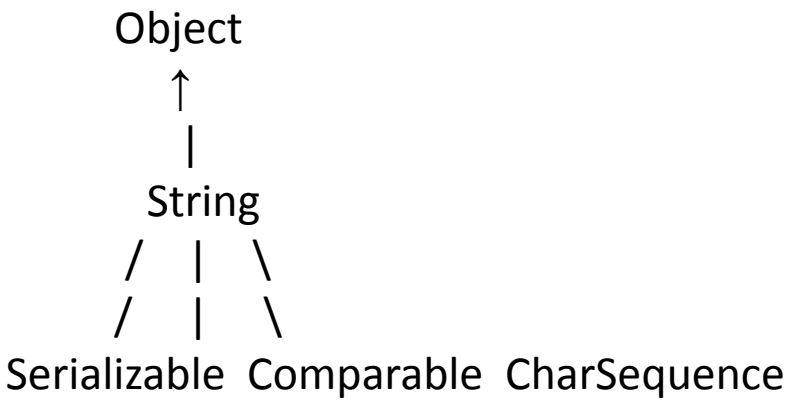
Method	Meaning	Example	Result
charAt(1)	Gets character at index 1	"Hello".charAt(1)	e
length()	Number of characters	"Hello".length()	5
subSequence(1,4)	Gets part of sequence	"Hello".subSequence(1,4)	ell
toString()	Returns String representation	"Hello".toString()	Hello

Class String:

- The class String is used to represent strings in Java. It is declared as

  public final class String extends Object

  implements Serializable, Comparable<String>, CharSequence



CONSTRUCTORS

No.	Constructor Prototype	Meaning	Simple Code Snippet
1	String()	Creates an empty String	String s = new String();
2	String(byte[] bytes)	Creates a String by decoding a byte array using the default charset	byte[] b = {65,66,67}; String s = new String(b); Output: ABC
3	String(byte[] bytes, Charset charset)	Creates a String by decoding bytes using the specified charset	byte[] b = {65,66,67}; String s = new String(b, StandardCharsets.UTF_8); Output: ABC
4	String(byte[] bytes, int offset, int length)	Creates a String from part of a byte array	byte[] b = {65,66,67,68}; String s = new String(b, 1, 2); Output: BC
5	String(byte[] bytes, int offset, int length, Charset charset)	Creates a String from part of a byte array using a specified charset	byte[] b = {65,66,67,68}; String s = new String(b, 1, 2, StandardCharsets.UTF_8); Output: BC

6	String(char[] value)	Creates a String from a character array	char[] c = {'H','e','l','l','o'}; String s = new String(c); Output: Hello
7	String(char[] value, int offset, int count)	Creates a String from part of a character array	char[] c = {'H','e','l','l','o'}; String s = new String(c, 1, 3); Output: ell
8	String(int[] codePoints, int offset, int count)	Creates a String from Unicode code points	int[] a = {72,101,108,108,111}; String s = new String(a, 0, 5); Output: Hello
9	String(StringBuffer buffer)	Creates a String from a StringBuffer	StringBuffer sb = new StringBuffer("Hello"); String s = new String(sb);
10	String(StringBuilder builder)	Creates a String from a StringBuilder	StringBuilder sb = new StringBuilder("Hello"); String s = new String(sb);

Methods for Extracting Characters from Strings:

No.	Method	Exact Prototype	Description	Simple Example	Result
1	charAt()	public char charAt(int index)	Returns the character at the specified index. Index starts from 0.	"Hello".charAt(1)	e
2	length()	public int length()	Returns the number of characters in the String.	"Hello".length()	5
3	getChars()	public void getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)	Copies characters from the String into a specified character array. srcBegin is included and srcEnd is excluded.	"Hello".getChars(1, 4, ch, 0)	ell
4	getBytes()	public byte[] getBytes()	Encodes the String into a sequence of bytes using the platform's default charset and returns the byte array.	"ABC".getBytes()	65 66 67
5	toCharArray()	public char[] toCharArray()	Converts the String into a new character array containing all its characters.	"Hello".toCharArray()	H e l l o

Methods for comparison of Strings

int compareTo(String str)	is used for comparing two strings lexicographically. Each character of both the strings is converted into a Unicode value for comparison. If both the strings are equal then this method returns 0 else it returns positive or negative value.
int compareToIgnoreCase (String str)	comparison of two strings irrespective of cases. Returns the value same as above.
boolean equals()	Compares the characters inside the two strings. It is different from ==(it checks references), where as equals() checks content.
boolean equalsIgnoreCase(String another)	Compares with another string. It doesn't check case. Returns boolean value.

boolean startsWith(String chars)	The startsWith() method checks whether a string starts with the specified character(s).
boolean endsWith(String chars)	The endsWith() method checks whether a string ends with the specified character(s).
public boolean regionMatches(int toffset, String other, int ooffset, int len)	It compares a selected portion of one String with a selected portion of another String.
public boolean regionMatches(boolean ignoreCase, int toffset, String other, int ooffset, int len)	

Methods for Modifying Strings:

No.	Method	Description	Example	Result
1	public String substring(int beginIndex)	Returns a substring from beginIndex to the last character.	"Hello".substring(2)	"llo"
2	public String substring(int beginIndex, int endIndex)	Returns a substring from beginIndex to endIndex-1.	"Hello".substring(1,4)	"ell"
3	public String concat(String str)	Concatenates the specified string with the existing string.	"Hello".concat(" Java")	"Hello Java"
4	public String replace(char oldChar, char newChar)	Replaces all occurrences of a character with another character.	"Hello".replace('l','x')	"Hexxo"
5	public String replace(CharSequence target, CharSequence replacement)	Replaces all occurrences of a sequence of characters with another sequence.	"Hello World".replace("World","Java")	"Hello Java"
6	public String trim()	Removes leading and trailing whitespace from the string.	" Hello ".trim()	"Hello"

Methods for Searching Strings

No.	Exact Prototype	Description	Simple Example	Result
1	public int indexOf(String str)	Returns the index of the first occurrence of the specified String.	"Hello Hello".indexOf("Hello")	0
2	public int indexOf(String str, int fromIndex)	Searches for the first occurrence of the String, starting from the specified index.	"Hello Hello".indexOf("Hello", 1)	6
3	public int lastIndexOf(String str)	Returns the index of the last occurrence of the specified String.	"Hello Hello".lastIndexOf("Hello")	6
4	public int lastIndexOf(String str, int fromIndex)	Searches backward for the last occurrence, starting at the specified index.	"Hello Hello".lastIndexOf("Hello", 5)	0

Class StringBuffer:

Constructors:

1. StringBuffer(): It reserves room for 16 characters without reallocation

StringBuffer s = new StringBuffer();

Capacity: 16

2. StringBuffer(int capacity): It accepts an integer argument that explicitly sets the size of the buffer.

StringBuffer s = new StringBuffer(20);

Capacity: 20

3. StringBuffer(String str): Creates a StringBuffer containing the specified String and provides capacity for **16 additional characters**.

StringBuffer s = new StringBuffer("PVPSIT");

- Capacity: 6+16 =22

- If we pass content to the constructor → capacity = 16 + content length.

1 StringBuffer()

Prototype

```
public StringBuffer()
```

Description

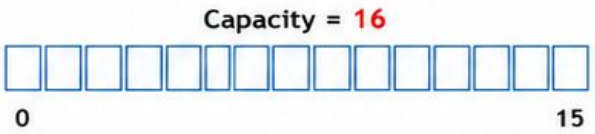
Creates an empty StringBuffer with a default capacity of 16 characters.

Example

```
StringBuffer sb =
  new StringBuffer();
```

- Initial content : "" (empty)
- Length : 0
- Capacity : 16

Visual Representation



No characters stored.
Buffer can hold 16 characters without reallocation.

2 StringBuffer(int size)

Prototype

```
public StringBuffer(int capacity)
```

Description

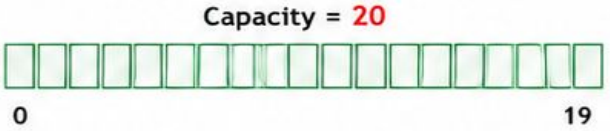
Creates an empty StringBuffer with the specified capacity.

Example

```
StringBuffer sb =
  new StringBuffer(20);
```

- Initial content : "" (empty)
- Length : 0
- Capacity : 20

Visual Representation



No characters stored.
Buffer can hold 20 characters without reallocation.

3 StringBuffer(String str)

Prototype

```
public StringBuffer(String str)
```

Description

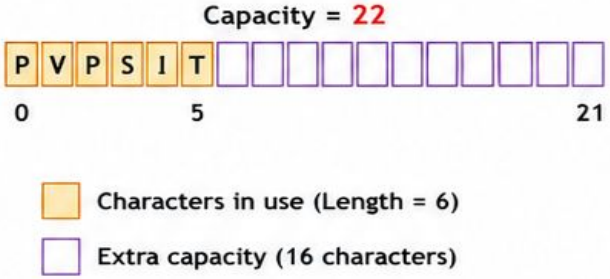
Creates a StringBuffer containing the specified String and provides capacity for 16 additional characters.

Example

```
StringBuffer sb =
 new StringBuffer("PVPSIT");
```

- Initial content : "PVPSIT"
- Length : 6
- Capacity : 22 (6 + 16)

Visual Representation



Sl. No	Method & Prototype	Description	Example	Result

1	**append(String s)**
	`public StringBuffer append(String s)`
		is used to append the specified string with this string. The append() method is overloaded like append(char), append(boolean), append(int), append(float), append(double) etc.	`StringBuffer sb = new StringBuffer("Hello");`
			`sb.append(" World");`
			`System.out.println(sb);`
				Output
				`Hello World`

2	**insert(int offset, String s)**
	`public StringBuffer insert(int offset, String s)`
		is used to insert the specified string with this string at the specified position.	`StringBuffer sb = new StringBuffer("Hello");`
			`sb.insert(5, " Java");`
			`System.out.println(sb);`
				Output
				`Hello Java`

3	**replace(int startIndex, int endIndex, String str)**
	`public StringBuffer replace(int startIndex, int endIndex, String str)`
		is used to replace the string from specified startIndex and endIndex.	`StringBuffer sb = new StringBuffer("Hello World");`
			`sb.replace(6, 11, "Java");`
			`System.out.println(sb);`
				Output
				`Hello Java`

4	**delete(int startIndex, int endIndex)**
	`public StringBuffer delete(int startIndex, int endIndex)`
		is used to delete the string from specified startIndex and endIndex.	`StringBuffer sb = new StringBuffer("Hello World");`
			`sb.delete(5, 11);`
			`System.out.println(sb);`
				Output
				`Hello`

5	**reverse()** `public StringBuffer reverse()`	is used to reverse the string.	`StringBuffer sb = new StringBuffer("PVPSIT"); sb.reverse(); System.out.println(sb);`	Output TISPVP
6	**ensureCapacity(int minimumCapacity)** `public void ensureCapacity( int minimumCapacity)`	is used to ensure the capacity at least equal to the given minimum.	`StringBuffer sb = new StringBuffer(); sb.ensureCapacity(50); System.out.println(sb.capacity());`	Output 50
7	**charAt(int index)** `public char charAt(int index)`	is used to return the character at the specified position.	`StringBuffer sb = new StringBuffer("PVPSIT"); System.out.println(sb.charAt(2));`	Output P
8	**length()** `public int length()`	is used to return the length of the string i.e. total number of characters.	`StringBuffer sb = new StringBuffer("PVPSIT"); System.out.println(sb.length());`	Output 6
9	**substring(int beginIndex)** `public String substring(int beginIndex)`	is used to return the substring from the specified beginIndex.	`StringBuffer sb = new StringBuffer("PVPSIT"); System.out.println(sb.substring(2));`	Output PSIT
10	**substring(int beginIndex, int endIndex)** `public String substring(int beginIndex, int endIndex)`	is used to return the substring from the specified beginIndex and endIndex.	`StringBuffer sb = new StringBuffer("PVPSIT"); System.out.println(sb.substring(1, 4));`	Output VPS

Remember

- Elaborate

  System.out.println()?

  public static void main(String ad[])

System.out.println()

Prints output to the console and moves the cursor to the next line.



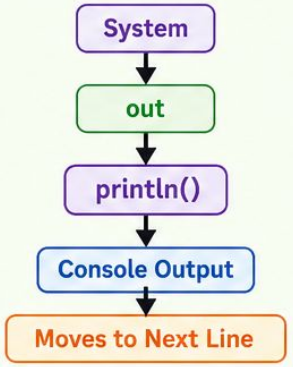
1. BREAKDOWN

Part	Meaning
System	Predefined class in java.lang package
out	Standard output stream (connected to console)
println()	Prints the data and moves cursor to the next line

2. SYNTAX

```
System.out.println(value);
```

- **value** → can be any data (String, number, character, boolean, object, expression)



public static void main(String[] args)

The main() method is the starting point of every Java program.

1 SYNTAX

```
public static void main(String[] args) {
    // statements
}
```

2 BREAKDOWN

Part	Meaning
public	Access modifier. JVM can call this method from anywhere.
static	Belongs to the class, not to any object. JVM can call it without creating an object.
void	Does not return any value.
main	Name must be main. JVM looks for this method to start execution.
String[] args	Can accept command line arguments as an array of String.

3 KEY POINTS

- It is the entry point of any Java application.
- The JVM searches for this method to start program execution.
- Without this method, the program will not run.
- It can receive command line arguments through args (array of String).

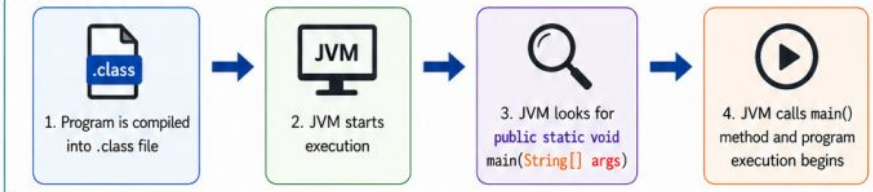
4 EXAMPLE

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
        System.out.println("Welcome!");
    }
}
```

OUTPUT

```
Hello, Java!
Welcome!
```

5 HOW JVM EXECUTES



Suresh Yadlapati